1. PROBLEM SOLVING FLOW ⚡

Problem Identification
↓
Analysis
↓
Algorithm
↓
Implementation
↓
Testing

Goal of Computational Thinking:

Solve problems systematically

---

2. TIME & SPACE COMPLEXITY ⚡

Time Complexity

Measures:

Growth of execution time with input size

Space Complexity

Measures:

Memory usage

IMPORTANT COMPLEXITIES ⚡

Complexity	Meaning
O(1)	Constant
O(log n)	Binary Search
O(n)	Linear
O(n log n)	Efficient sorting

Complexity	Meaning
$O(n^2)$	Nested loops

---

3. ARRAY ⚡

- Linear Data Structure
- Fixed size
- Fast indexing

Access:

$$O(1)$$

---

4. LINKED LIST ⚡

- Dynamic size
- Nodes connected using pointers
- Slower access than array

Types:

- Singly
- Doubly
- Circular

Circular Linked List:

Last node points to first node

Advantage over array:

Dynamic size

---

5. STACK ⚡

Principle:

LIFO

Operations:

- Push
- Pop
- Peek

Used in:

- Recursion
- DFS
- Undo
- Expression evaluation

Stack Overflow:

Infinite recursion / too many pushes

---

6. QUEUE ⚡

Principle:

FIFO

Operations:

- Enqueue
- Dequeue

Used in:

- Scheduling
- BFS

---

7. CIRCULAR QUEUE ⚡

Advantage:

Reuses freed space

---

8. TREE ⚡

Non-linear data structure.

---

9. BINARY TREE ⚡

At most 2 children

---

10. BST (Binary Search Tree) ⚡

Rule:

Left < Root < Right

Inorder traversal:

Gives sorted output

Worst case:

Skewed tree → O(n)

Balanced BST:

$$O(\log\ n)$$

---

11. AVL TREE ⚡

Self-balancing BST

Maintains:

Balance Factor

Uses:

Rotations

Purpose:

Maintain balanced height

---

12. HEAP ⚡

Complete Binary Tree

Types:

- Min Heap
- Max Heap

Used in:

Priority Queue

Heap insertion:

$$O(\log\ n)$$

Heap construction:

$$O(n)$$

Heap sort:

$$O(n\log\ n)$$

---

13. HASHING ⚡

Purpose:

Fast searching

Average search:

$$O(1)$$

Collision:

Two keys map to same index

Collision resolution:

- Linear probing
- Quadratic probing
- Double hashing

Linear probing:

Check next slot sequentially

---

14. RECURSION ⚡

Function calls itself.

Must have:

Base condition

Otherwise:

Infinite recursion

Uses:

Call Stack

Tail recursion:

Recursive call is last operation

Tail recursion optimization reduces:

Stack usage

---

15. SEARCHING ⚡

Linear Search

$$O(n)$$

Binary Search ⚡

Needs:

Sorted array

Technique:

Divide and Conquer

Complexity:

$$O(\log\ n)$$

Fails on:

Unsorted array

Worst case:

Element not found

---

16. SORTING ⚡

Bubble Sort

- Stable
- Adjacent swaps
- Brute force

Worst:

$$O(n^2)$$

Selection Sort

- Select minimum element
- Usually NOT stable

Worst:

$$O(n^2)$$

Insertion Sort

Best for:

Nearly sorted data

Stable + In-place

Merge Sort ⚡

Technique:

Divide and Conquer

Properties:

- Stable
- Recursive
- Uses extra memory

Complexity:

$$O(n\log\ n)$$

Recurrence:

$$T(n) = 2T(n/2) + n$$

Extra space:

$$O(n)$$

Quick Sort ⚡

Uses:

Pivot

Average:

$$O(n\log\ n)$$

Worst:

$$O(n^2)$$

Worst case:

Unbalanced partition / bad pivot

Properties:

- In-place
- Usually NOT stable

---

17. GRAPH ⚡

Graph contains:

- Vertices
- Edges

Types:

- Directed
- Undirected
- Weighted

---

18. GRAPH REPRESENTATION ⚡

Adjacency Matrix

Best for:

Dense graphs

Space:

$$O(V^2)$$

Adjacency List

Best for:

Sparse graphs

Space:

$$O(V + E)$$

---

19. BFS vs DFS ⚡

BFS	**DFS**
Queue	Stack
FIFO	LIFO
Level-wise	Depth-wise
Shortest path in unweighted graph	Deep exploration

Complexity:

$$O(V + E)$$

DFS recursion uses:

Call stack

---

20. SHORTEST PATH ⚡

BFS

Used for:

Shortest path in unweighted graph

Dijkstra ⚡

- Greedy algorithm
- Uses Priority Queue
- Uses edge relaxation

Finds:

Shortest path

Fails when:

Negative edges exist

Bellman Ford ⚡

Handles:

Negative weights

Slower because:

Relaxes edges multiple times

Floyd Warshall ⚡

Finds:

All-pairs shortest path

Technique:

Dynamic Programming

Complexity:

$$O(V^3)$$

---

21. MST (Minimum Spanning Tree) ⚡

Prim's Algorithm

- Greedy
- Vertex-based
- Better for dense graphs
- Uses Priority Queue

Kruskal's Algorithm

- Greedy
- Edge-based
- Better for sparse graphs

Uses:

Disjoint Set Union

---

22. GREEDY ALGORITHM ⚡

Idea:

Choose locally optimal solution

May NOT always give global optimum.

Examples:

- Dijkstra
- Prim

- Kruskal

---

23. DYNAMIC PROGRAMMING ⚡

Properties:

- Overlapping subproblems
- Optimal substructure

Purpose:

Avoid recomputation

Memoization

Top-down DP

Tabulation

Bottom-up DP

Examples:

- Fibonacci
- Knapsack
- LCS

Naive Fibonacci:

$$O(2^n)$$

DP Fibonacci:

$$O(n)$$

---

24. BACKTRACKING ⚡

Idea:

Try → Undo → Retry

Like:

DFS with pruning

Examples:

- N Queens
- Sudoku

---

25. TOPOLOGICAL SORT ⚡

Applies only to:

DAG

Usually uses:

DFS + Stack

---

26. IMPORTANT TRUE/FALSE RAPID FIRE ⚡

Stack → LIFO
Queue → FIFO
DFS → Stack
BFS → Queue
Heap → Complete Binary Tree
AVL → Self-balancing BST
Binary Search → Sorted array
Quick Sort → Pivot
Merge Sort → Stable
Quick Sort → Usually unstable
DP → Stores answers
Greedy → Local optimum
Backtracking → Try and undo
BST inorder → Sorted order

---

27. MOST IMPORTANT COMPLEXITIES ⚡

Algorithm	Complexity
Linear Search	O(n)
Binary Search	O(log n)
Bubble Sort	$O(n^2)$
Selection Sort	$O(n^2)$
Merge Sort	O(n log n)
Quick Sort Avg	O(n log n)
Quick Sort Worst	$O(n^2)$
BFS	O(V+E)
DFS	O(V+E)
Floyd Warshall	$O(V^3)$

FINAL MEMORY BOOST ⚡🔥

BFS → Queue
DFS → Stack
BST inorder → Sorted
AVL → Balanced BST
Heap → Priority Queue
Quick sort → Pivot
Merge sort → Stable
Binary Search → Sorted array
DP → Memoization
Greedy → Local best
Backtracking → Try + Undo